Temco Platform

Extension design · built Nov 12, 2026

Extension design

> **Layout superseded.** The views, the Pages tree behaviour and the flow are defined in ../../ui/redesign.md. Files such as `home.ts` and `views.ts` mentioned below do not exist; the code is under `extensions/temco-docs/src/`.

Responsibilities, one file each

Module	Owns
`src/registry.ts`	reading `docs/help/registry.json` and the generated `index.json` / `page-meta.json`, the page id → file rule, the structural checks, the counts. No VS Code UI beyond `Uri`, `WorkspaceFolder` and the filesystem
`src/tree.ts`	the Content view: which nodes exist, what they show, what a click does
`src/diagnostics.ts`	which issues become Problems, and on which file
`src/extension.ts`	activation, the view and status bar, the commands, the file watcher, and the model it publishes to other modules (`src/api.ts`)
`src/store.ts`	what this module reads of `temco-store` — the catalog, the selection, the slug rule — in one file, and the only place that crosses modules
`src/ui.ts`	what it reads of `temco-ui` — where a page of ours opens, and what a command argument names
`src/projects.ts`	what it knows of `temco-projects` — one signal, so the Library's rows come back when a project changes
`src/repositories.ts`	what it reads of `temco-repositories` — a source's code files, the material for a draft, and the repository's own scripts
`src/api.ts`	what this module publishes: the documentation model `temco-ai` reads (the sets, the pages, the selection, the editor column, one change signal)
`src/aiModule.ts`	what this module knows of `temco-ai` : whether it is installed, its commands, and the two calls it makes when it is (a draft, an analysis)
`src/builds.ts`	the product repo's own builds, invoked as VS Code tasks and judged by the artifact they leave — never by an exit code, and never the form that deploys
`src/report.ts`	the staleness report: what needs attention, grouped by owner, as Markdown. Counts rather than lists for the pages that have merely never been reviewed
`src/scaffold.ts`	starting a registry in an open folder that has none: the smallest valid file, on a confirmation, with the seeding left to the repository's own generator
`src/home.ts`	the dashboard: the one page that is not about a folder — identity, how the application works, what exists now, how to start, and every feature with its state
`src/views.ts`	the other two sidebar sections: **Work** (what needs doing, and what changed) and **Library** (what it reads and produces)
`src/feedback.ts`	feedback rows routed to their page's owner, against a proposed file shape, with the aging called out
`src/previewPanel.ts`	the page as a reader sees it, in its own editor tab beside the source: one panel per document, redrawn as the page is typed. Rendering is the built-in Markdown pipeline, so the preview and the editor's own idea of the page cannot disagree; the DocFX `> [!NOTE]` markers are dressed as the alerts the site publishes

Module	Owns
`src/diffPage.ts`	the page against its last commit, in `vscode.diff`. HEAD's text is read with `git show`, written under the extension's storage, and deleted when the tab closes. The right-hand side is the editor's buffer, so an unsaved change is part of the diff it shows
`src/outline.ts`	the **Page** pane: the page's headings as a nested jump list, and its word count in the status bar. Fenced blocks are skipped, because a `#` inside a code sample is a comment
`src/fluentEditor.ts`	**Temco Markdown**: the authoring surface — the page as a document on Fluent UI, with capture and annotation in the surface. The default editor for `*.md`; it owns the view type and the open command, and nothing of the protocol
`src/markdownEdits.ts`	what bold, a list, a heading, a table and an image **mean**, as pure transforms over text and offsets — plus the writing behaviours (`continueList`, `indentLines`) and the search (`findMatches`, `replaceAll`). Both editors call it — Monaco through `src/format.ts`, the Fluent surface directly — so the two cannot drift into producing different files
`src/attachments.ts`	where an image lands: the `media/` folder, the `NN-slug` name, the size limit, the width warning, and the record beside it. One path for the Monaco command, the surface's paste, a capture and a file picked from disk
`src/attachmentRecord.ts`	that record as data — parse, serialise, add a caption, decide what is stale — with no `vscode` import, which is what makes the rule testable
`webview/docs/main.tsx`	that surface's shell, its caret rules, its paste matrix, its caption question, its outline, its find bar and its view modes; `annotate.tsx` is the canvas annotator beside it, `highlight.ts` the colour behind the text, `find.tsx` and `outline.tsx` the two panels, `theme.ts` the `--vscode-*` → Fluent token bridge, `markdown.ts` the page renderer
`scripts/check-fluent.mts`	the content-library boundary: only `webview/docs` may import `@fluentui`. `npm run check-fluent`
`scripts/check-docs.mts`	the Fluent surface's bundle loaded into a DOM and driven the way the host drives it — it renders, `ready` is posted **once**, the formatting actions produce the right Markdown, an identical `set` disturbs nothing, an outside change still arrives. `npm run check-docs`
`scripts/check-record.mts`	the attachment record through a table of cases, including the one that matters: a picture with no recorded version is never called stale. `npm run check-record`
`tsconfig.webview.json`	the webview sources type-checked on their own, since `tsconfig.json` covers the extension host only. `npm run check-types`
`scripts/check-learn.mts`	the Temco Learn portal: what a `manifest.json` means, and what the page it writes says. The first coverage of a file written on every publish and read by nobody. `npm run check-learn`
`src/learnPage.ts`	the portal page and the manifest rules, as pure functions — no `vscode`, so the publish path has something to test
`src/webviewText.ts`	the bridge every webview surface shares: one `WorkspaceEdit` per save, line endings preserved, the echo guard, the capture handshake, and the bar actions (preview / diff / source / properties). Nothing that could corrupt a file is decided twice

Module	Owns
`src/csv.ts`	CSV as a grid can hold it and back again: RFC 4180 quoting, delimiter and end-of-line detected, and the *quoting style* preserved per cell so a one-cell edit is a one-line diff. No VS Code here at all
`src/gridEditor.ts`	**Grid mode**: a CSV edited as cells (Univer). Same bridge as `webviewText.ts`; the file stays line-per-row because `csv.ts` does the writing
`scripts/build-webview.mts`	the one bundler step: `webview/grid.ts` → `out/webview/grid/` and `webview/docs/main.tsx` → `out/webview/docs/`. `tsc` cannot do this — the surfaces are browser code with npm dependencies

It no longer has a `src/catalog/` or a `src/selected.ts`: the database, the selection and the slug rule are `temco-store`'s, reached through `src/store.ts`.

`registry.ts` is deliberately free of presentation: it is the part that would survive replacing the view, and the part any other module reuses to know what pages exist — `temco-ai` reads it through the exported model rather than through this folder.

The tree

TEXT

```
Content                       (view container: Temco Docs)
├─ Gaps (12)                  only when something is wrong, across all open repos
├─ T3000Webview-EEZ-Studio    only when more than one folder is open
│  ├─ t3000                   title · audience · noindex
│  │  ├─ api-reference        chapter, with a page count
│  │  │  ├─ Data Points API    owner · ship · reviewed, plus its issues as children
│  │  │  └─ …
│  │  └─ …
│  └─ legacy
└─ …
```

- Counts live on the **parents**, because "which area needs attention" is the real question.
- A page with issues is collapsible and its children are the reasons.
- A page's tooltip carries what the generator knows and the registry does not: its last change, its headings, and — marked in the description and the icon — whether its `code` moved on since the last review. With `index.json` / `page-meta.json` absent, that block is simply missing and nothing else changes.
- Clicking a page opens the file; if the file does not exist, the click offers to create it.
- Areas and chapters are never empty: an area declared in `areas{}` with no pages still appears, which is itself information.

The page in the editor

The editor of record is **VS Code's own**, deliberately. `15-online-editor.md` rejects a rich editor for these files on a reason that survives checking: the pages are written one paragraph per source line (measured: 95 characters on

average, 168 lines over 200), so an editor that re-wraps rewrites every paragraph it touches and a one-word change becomes a whole-file diff. What was missing was never the engine — it was the chrome around it.

Surface	What it is	Why not something new
the toolbar	`src/format.ts`, on `editor/title` and in the context menu	Monaco has no formatting bar; the buttons edit the buffer and undo like typing
Preview	`previewPanel.ts`, a tab beside the source (`preview:<page>.md`)	VS Code shows a page, the build shows an alert; the panel dresses the markers so the two agree
Diff	`diffPage.ts`, `vscode.diff` against `HEAD`	the editor's own diff, so the hunks seen are the hunks the pull request carries
the **Page** pane	`outline.ts`: headings as a nested jump list, plus a word count in the status bar	line, column, encoding and end-of-line are already the status bar's, so nothing repeats them
Grid	`gridEditor.ts` + `webview/grid.ts`: a CSV as cells, reached by *Open in the Grid*	the file stays CSV, one row per line, so a one-cell edit stays a one-line diff
Temco Markdown	`fluentEditor.ts` + `webview/docs/`: the page as a document on **Fluent UI**, with capture and annotation built in — the **default** for `*.md`	see *The Fluent Markdown surface* below
Temco Editor	not this module any more: the page editor is `extensions/temco-editor` (`temcoEditor.markdown`), and **Open in Temco Editor** asks for it by command	see temco-editor

One bridge, two surfaces

> **Record.** This is the period when `temco-docs` shipped two editing surfaces at once. It ended: `Temco Markdown` is gone, the page surface became its own module, and the shared bridge went with it (temco-editor). What is left here is only the grid.

Two editors over one file is a decision, not an accident: **what you see while editing** is a real choice, and it cannot be both. A source-visible editor (`Temco Markdown`) is the only way to be sure `git diff` will show what you did; a WYSIWYG editor (`Temco Editor`) is the only way to write a table without thinking about pipes. Everything *else* about them is shared, and it is shared in code, not by convention:

Shared, in the bridge (`src/webviewText.ts` , now `temco-editor/src/bridge.ts`)	Why it cannot be per-surface
the ordered-write guard (a set of texts asked for, not just the newest)	a write that lands late looks like an outside change, and answering it moves the caret to the end of the page
the file's own line endings	a browser only has `\n` ; writing that into a CRLF file is a whole-file diff for one word
one `WorkspaceEdit` over the whole file	so `Ctrl+Z` undoes an edit rather than a keystroke, and the dirty dot means the same thing
the capture pipeline (`image` , `pastedImage` , `capture` , `commitCapture` , `caption` , `linkPage`)	the picture lands beside the page with its words and its version; two copies of that would drift
the surface-failure reporter in the HTML	an editor that cannot start must say so in its own tab — an empty one is the failure a person cannot act on

`PageEditor` is therefore 30 lines: it names the bundle (`page`) and nothing else. Two surfaces, one set of rules. The **ribbon** is the second half of that: `.ribbon` lives in `temco-ui/media/page.css` , so both editors wear the same strip.

A surface that asks a question draws the question itself. A webview frame is sandboxed without modals, so `window.prompt` returns `null` there — the Link button was dead for exactly that reason, and the page now asks in the vocabulary's own box (`.page-surface__ask`), where Enter accepts and Escape cancels.

The controls both surfaces need live in `webview/shared/` : `find.tsx` (the bar; replacement is optional, because a rich document cannot truthfully "replace text" across formatting), `outline.tsx` (a jump list whose target is called `at` — a line for a source surface, a document position for a rich one), `annotate.tsx` (the canvas annotator) and `surface-controls.css` . They were inside `webview/docs/` until the page needed them; a copy in each surface is how the two would drift.

The page's chrome: a document application, not a card

The rich page is arranged the way a document application is, because that is what it is: `Home · View` at the top, groups under each, a sheet on a canvas, and a status strip. Two tabs and not three: there are about twenty controls, and the insert ones (link, picture, screenshot, table) are part of writing, so they sit on `Home` beside the marks and the block commands. `View` is what reads the page or hands it to another surface. A third tab for six controls was a tab nobody visits, and it made every control a guess about which of the two tabs held it.

Element	Rule	Why
the ribbon	flush to the tab: no border on three sides, no corners, `border-bottom` only	a card floating in a window is a page; a document's ribbon is part of the window
the controls	**30 px icon buttons, 20 px glyphs, 44 px panel**, no group captions	a writing bar should cost as little vertical space as possible; the captions are dropped because a caption under every group is a teaching aid a narrow editor pane cannot afford — the hairline between groups and each control's tooltip carry the same information. Only an icon-only control gets a square (`btn--icon`); a word button keeps its own width, which is what stops two of them drawing on top of each other
density	`full` ≥ 940 px, `compact` below — measured, not guessed	905 px is what the whole `Home` tab needs (83 history + 215 marks + 128 style + 128 lists + 149 blocks + 182 insert + hairline and padding). A compact bar keeps nine controls and folds five into `⋯`. Every group declares its controls **once**, with a tier, and `⋯` is the same table with the tier-2 rows taken out — an overflow menu written by hand is a second copy that drifts
the sheet	`.page-surface__paper` : **no border, no radius**, on `.page-surface__canvas` , `min-height: calc(100% - 16px)`	a sheet is a sheet; the canvas is a plain scroll container, **not** a flex row — a flex item is stretched to the container's height, which cut the sheet off at the fold and let the rest of the document spill onto the grey
one paragraph style	the **Text** menu names the block the caret is in and sets it; there is no second row of `Normal / Heading 1–3` buttons	two controls for one thing made the bar teach two ways to do it, and the buttons pushed the insert group off a narrow pane
zoom	`– 100% +` in the status strip, a view only	it never reaches the file, which is why it can exist at all
Find / Outline	Find is on the **View** tab; the outline is a **rail on the right** with its toggle at the far right of **Home** and `Ctrl+Alt+O` anywhere	navigating a document is not editing it — but a table of contents is *used* while writing, so its toggle lives where the writing is. Right, not left: the rail is the TOC the published page shows, so the structure you edit is the structure you ship
the outline as a tree	nesting from `src/outlineModel.ts` (`scripts/check-outline.mts` proves it): a heading hangs off the nearest one above it with a lower level, **however many levels are skipped**; the row the caret is in is marked current; folding is per row and remembered; the filter keeps the branch that leads to a match	a list of indented headings cannot say "this belongs to that", and a `###` after a `#` is a real document, not an error. The model is pure so the three rules that fail silently — a skipped level, a branch that springs open while typing, a caret with no current row — are checked without a browser

Element	Rule	Why
the outline rail	**300 px by default**, docked on the right, **dragged by its divider** (180–560 px, arrow keys too) and remembered in the webview state; never floated	the width is a preference, so it is set by hand once rather than guessed at; the two limits are what stop a drag from eating the page. It is `Ctrl+Alt+O` away if it is in the way
a toggle that is on	the glyph takes the **accent**, a hairline ring says pressed — never a filled block	a block of the selection colour reads as a button that was just clicked, not as a mode that is *on*. Bold, italic, Find, Outline and the rail all use it, so an "on" looks the same wherever it is
the rail opens itself	on a page with **three or more** headings, once, and never again for someone who closed it (`outline: undefined` means "never asked")	a TOC of one line is furniture; a rail that reopens itself after being closed is a fight
Read mode	View → *Read the page*: a second engine (`reading.tsx`) over the same seam, no caret, no editing tabs	reading a page should not be one keystroke from changing it; and it is the proof that the seam is real
Link · Picture · Screenshot · Table	the **Home** tab, beside the marks: a link and a picture are what writing a page is made of	Word's `Insert` tab exists for the fifty things Word can insert; six do not earn a tab. The horizontal rule, which used to be in two tabs, is now in one place
Annotate	**Home**, enabled only when a picture is selected, and it **replaces** that picture	the page keeps one figure, not a pair

What is deliberately **not** in this ribbon: font, font size, text colour, alignment, spacing, margins. Markdown cannot hold them, and a control whose effect disappears on the next save is a lie — the same reason the alignment group was removed from the source surface.

The editor's three files, and the seam between them

```TEXT
webview/page/
  main.tsx       409  the mount: host protocol, the document, find/outline/zoom/status
  ribbon.tsx     311  the bar: tabs, groups, gallery, overflow      ← imports no Tiptap
  document.tsx   297  the engine: extensions, usePageDocument, actionsFor, stateOf
  reading.tsx     50  a second engine that cannot edit             ← proves the seam
```

> **Where this is now.** The three files moved to `extensions/temco-editor/webview/markdown/` when the editor became its own module, and the seam was separated from the engine: the two types below live in `contract.ts` (no package named in it), and `document.tsx` sits under `engine/tiptap/` with `bubble.tsx`, `handle.tsx` and `taskLists.mts`. Everything this section says about the seam still holds; only the paths changed. `temco-docs` keeps `webviewText.ts` ' share of the work as `src/webview.ts`, which is now just `webviewAssets` for the grid.

The seam is two types, both in `document.tsx`:

`PageState`	what is true at the caret — `block`, `marks`, `lists`, `inTable`, `onImage`, `link`, `canUndo`, `canRedo`, `words`
`PageActions`	named operations — `setBlock`, `toggleMark`, `setLink`, `insertImage`, `replaceImage`, `goto`, `load`, `content`, `node`…

`grep -r "@tiptap" webview/page/ribbon.tsx webview/shared/` returns nothing, which is the property worth keeping: the bar can be re-skinned without touching the engine, and the engine replaced without touching a button. Read mode is the proof — it hands the same `PageRibbon` a frozen `READING_ACTIONS` and static markup instead of an editor, and the bar draws itself exactly as before with its editing tabs removed.

One trap the split exposed, recorded because it will catch the next person: an actions facade built on every render makes the host-message effect re-subscribe on every keystroke, and the document that arrived while the effect was detached is lost — the page then opens with stale text. The facade is one object per editor (`useMemo` on the editor instance).

The three optional surfaces are webviews, so they need a bundler — the only one in this extension:

```
TEXT

webview/grid.ts        ──esbuild──▶  out/webview/grid/grid.js      11.9 MB  Univer (Apache-2.0)
```

The page surface has its own bundler in its own module (`extensions/temco-editor`, `out/webview/markdown/`), because a module builds what it owns.

Where this belongs: the extension, and what moving it into core would take

The surfaces are an extension today. The question "should this be product code instead" has a factual answer in this fork, so it is recorded here rather than argued:

	Extension (`extensions/temco-docs`)	Core (`src/vs/workbench/contrib/…`)
how a surface is registered	`customEditors` contribution + `registerCustomTextEditorProvider`	an `EditorPane` on `EditorExtensions.EditorPane` with its own `EditorInput` + `IEditorSerializer`; the in-tree templates are `contrib/imageCarousel` (pane + webview) and `contrib/update/releaseNotesEditor` (a webview editor)
what it can reach	the public API: `vscode.*`	every service: `IEditorService`, `IStorageService`, `IFileService`, `IContextKeyService`, `INotificationService`, the chat/agent services, `MenuRegistry`, `ViewsRegistry`
tab restore	`retainContextWhenHidden`, a webview kept alive in a hidden tab	the editor input serializes itself, so a tab comes back without the hack
iteration	edit, `npm run build-webview`, reload the window	edit, client compile, repo gates
price of a change	local and quick	`valid-layers-check`, layering rules, a full client compile

Two things decide it. **A core surface cannot `import` React, Tiptap or Fluent** — `src/vs` has no React and no JSX toolchain — so the *content* would still be a webview bundle whatever we do; only the *host* moves. And the fork already contains the pattern for that split (`imageCarousel` : DOM in core, a webview for what needs one).

The target, when the surfaces stop being experimental:

```
TEXT
```

```
src/vs/workbench/contrib/temcoDocs/
  browser/
    temcoDocs.contribution.ts        registers the panes, commands, menus, views
    markdownPageEditor.ts            EditorPane   ┐
    markdownPageEditorInput.ts       EditorInput  ├ the authoring surface
    pageEditor.ts                    EditorPane   ┐
    pageEditorInput.ts               EditorInput  ├ the document surface
    media/docs.js · media/page.js    built bundles (esbuild, not committed)
  common/                            the round trip, the page model, the registry
  test/browser/                      what `check-page` and `check-docs` do today
```

The migration is worth doing when one of these is true: a surface needs a workbench service an extension cannot see (the editor group layout, the chat context, a real `IStorageService`), tab restore has to be exact, or the module needs its own keybindings and menus without the `when` -clause traps. Until then the extension is not a limitation being tolerated — it is the cheapest place to keep a surface that is still being designed, and everything above it (the round trip, the capture pipeline, the page model) is already plain TypeScript that moves without a rewrite.

The output goes to `out/webview/` , beside the extension host JavaScript and inside the directory this repository already treats as build output, so it is ignored by git. It is deliberately **not** committed: the `local/code-no-new-javascript-files` rule rejects a newly added `.js` , and only a CODEOWNERS edit can allowlist one. `vscode:prepublish` runs the build for a packaged extension, so a source tree needs `npm run build-webview` once.

What was tried and removed: the rich editor

The page was once offered twice: the source surface, and a **Rich mode** built on the official Tiptap UI Components template — 161 vendored files, 21 @tiptap/* packages and a 1.6 MB bundle. It was **removed**, because:

- **The file is the source of truth, and it is Markdown.** The site, the offline help and the AI context all read it, and review is a git diff of one paragraph per line. A ProseMirror document has to be serialised back, and a round trip that re-wraps a paragraph turns a one-word edit into a whole-file diff — which is the failure this whole document exists to avoid.
- **It cost more than it gave.** Four extensions had to be removed because they do not survive that round trip (TextAlign, Subscript/Superscript, Typography, colour highlight), each one a standing risk that the repairs in `tidy()` cannot keep up. Two editors was also two sets of interactions to keep right, and the one that showed the document *as a document* was never the one that showed the bytes.
- **It did not look like the product.** It wore the template's own chrome rather than `page.css` , so the two editors did not resemble each other, let alone the rest of the application.

What it taught is kept, because it is the reason the shipped surface has the caret rules it has: a set that replaces the document under the caret moves the caret to the end of the page, an echo of the editor's own write must be dropped rather than applied, and `applyEdit` landing out of order makes a late write of ours look like an outside

change. Those three lessons are enforced in `webview/docs/main.tsx` and asserted in `check-docs` and in the scenario below.

The rich page: Tiptap, and the round trip that makes it safe

> **Where it lives now.** This surface became its own module: `extensions/temco-editor`, view type `temcoEditor.markdown`, and it is the **default** for `*.md` today. The paths below are the ones it had while it lived here; the reasoning, the rules and the checks are unchanged and still apply — see temco-editor for the current tree. The two-surface comparison this section describes is over: the winning affordances (a selection menu, a block handle, a placeholder) are in that module, the losing prototype was deleted, and the ribbons of the two surfaces became one.

The rich editor is back, and it is the same engine — Tiptap, then 3.30.3 and now 3.32, twelve packages, its own bundle at 983 KB (1457 KB today, with Fluent, Radix and the drag handle).

Part	What it was	Where it is now
`src/pageMarkdown.ts`	the whole contract: `toHtml` (`marked`), `toMarkdown` (`turndown` + GFM), `roundTrip`	`temco-editor/src/markdown.ts`
`webview/page/main.tsx`	the Tiptap editor and its ribbon; it never sees a file, only Markdown over the message port	`temco-editor/webview/markdown/main.tsx`
`webview/page/taskLists.mts`	Tiptap's `TaskList` / `TaskItem`, extended to accept the shape `marked` writes as well as their own	`temco-editor/webview/markdown/engine/tiptap/taskLists.mts`
`pageEditor.ts`	`temcoDocs.page`, a `CustomTextEditorProvider` on the same text document the source surface edits	`temco-editor/src/surfaces/markdown.ts` (`temcoEditor.markdown`)
`scripts/check-page.mts`	thirteen cases, one of them a whole page that must come back byte-identical	`temco-editor/scripts/check-page.mts`
`scripts/check-task-lists.mts`	the two task-list shapes, through a real editor in jsdom: open, save, and the lists that must not be claimed	`temco-editor/scripts/check-task-lists.mts`

The rules the conversion needs, each one a case in `check-page`:

- Tiptap wraps a table in `<colgroup>`; the GFM rule only recognises a table whose first child is a row, so a table would serialise as raw HTML and the file would lose it. The colgroup is dropped — column widths are the reader's.
- A paragraph inside a list item or a table cell emits **no** blank line. With one, a tight list becomes loose and every bullet is rewritten, and a table row breaks into fragments.
- Turndown pads a bullet marker to the width of the longest marker in the list (`- one`), so markers are collapsed back to one space. A task item's checkbox is followed by one space, not two.
- `*italic*` is written `*italic*`, not `_italic_`; the GFM plugin's strikethrough writes a single `~`, which is not strikethrough at all, so it is overridden to `~~`.
- Underline and highlight have no Markdown spelling. They are kept as the inline HTML Markdown already allows (`<u>`, `<mark>`), because a button whose effect disappears on the next save is a lie.
- **A task list is two shapes, and both are accepted.** `marked` writes `- [x] done` as `<ul><li><input type="checkbox" checked> done</li></ul>`; Tiptap renders it as `ul[data-type="taskList"] > li[data-type="taskItem"]` with the checkbox inside a `<label>`. Without a rule for each direction the checkbox was lost twice — the page opened as a plain bullet list, and an item that did survive saved as `- done`. `engine/tiptap/taskLists.mts` adds the second shape to the parser, and the `taskCheckbox` rule in `src/markdown.ts` reads the checkbox wherever it sits. `check-task-lists.mts` drives a real editor over both shapes; `check-page.mts` keeps the serialised result byte-identical.
- **Alignment and colour are not in the toolbar.** A paragraph's alignment lives in a `style` attribute Markdown cannot hold, which is the same reason table widths are not kept. The reader decides both.

The two surfaces are peers over one file, not two products: the tab's menu reaches the other one (`temcoDocs.page.open`, *Open With…*), and both write the same document, so save, the dirty dot, `Ctrl+Z` and `git diff` mean the same thing in either. The Markdown source surface stays the default for `*.md`; a workspace that wants the page instead changes one `priority` in `package.json`, or a person picks it once in *Open With…*.

The two editors also share one ribbon: `.ribbon` and its groups live in `temco-ui/media/page.css`, so the strip a writer recognises in the source surface is the strip in the page, drawn from the same buttons and badges.

The engine and the design language: both stay

Two questions come back often enough to answer once.

Tiptap is not the editor; it is the document model. It owns the schema, the text, the caret, the undo history and the HTML — `engine/tiptap/document.tsx` (the schema and the editor) and `engine/tiptap/taskLists.mts` (the two task-list shapes) are the only files in the product that import `@tiptap/*`. Everything that a person sees or presses is ours: `ribbon.tsx`, `main.tsx`, `outline.tsx`, `find.tsx`, and the round trip in `src/markdown.ts`, which converts HTML ⇄ Markdown with `marked` / `turndown` and has no Tiptap import at all. The contract between the two is `PageState` (a snapshot) and `PageActions` (named operations), and `reading.tsx` runs the same ribbon, outline and status strip behind an implementation with **no editor**, which is the seam being real rather than drawn. Since the move, the contract is its own file (`webview/markdown/contract.ts`) and names no package at all.

Swapping the engine is therefore possible and pointless: it would rewrite those 315 lines, change no button, and re-prove the behaviour `check-page` already proves. Tiptap earns its place at roughly a third of the bundle, because a rich-text model with tables, task lists, images, marks and undo is not a weekend's work.

Fluent is not the design language either. It supplies the **icons** (plain SVGs, no provider needed, five files) and **one dialog** — the annotator, ten Fluent components plus the `FluentProvider` that gives it the editor's light or dark

theme. The ribbon, the menus (Radix, for positioning), the outline tree, the sheet and the status strip are the vocabulary's own HTML and CSS. `scripts/check-fluent.mts` fails the build if anything outside `webview/markdown`, `webview/shared` (in `temco-editor`) imports it, so a second design language cannot creep in unnoticed. Removing Fluent entirely would mean replacing those glyphs and one dialog — worth doing when the annotator next needs a change, not before.

What was removed: ONLYOFFICE

`src/office/` (sign, config, bridge, desktop paths, the custom editor), its four commands, its settings block and `scripts/check-office.mts` were deleted. ONLYOFFICE Docs does edit `.md` natively and the integration worked, but the licence and the deployment each decide the question on their own:

- **AGPL-3.0.** The Document Server is a network service; serving it to our own users is exactly the case the licence covers, and embedding it in a desktop product does not avoid that.
- **A server nobody has.** It is not a library — it is a Docker or WSL deployment with its own storage, callback URLs and JWT signing. No such server exists in this repository or on the machines this product runs on, and a document editor that needs one before it can open a file is not a desktop product.

Tiptap answers the same need — a document, in the product, offline, on the file — with a licence we can ship.

Importing Word documents

`src/docx.ts` + `src/docxConvert.ts`: `mammoth` reads the `.docx` into semantic HTML and `turndown` turns that into Markdown, in a child process so the extension host never blocks. The result is a *starting point* — Word's fonts, colours, spacing and text boxes have no Markdown equivalent and are dropped rather than faked — which is why it writes a new page beside the document instead of replacing anything.

The converter is TypeScript, compiled to `out/docxConvert.js` and spawned from there. It is deliberately not a `.mjs` beside the sources: the repository's pre-commit hook rejects a newly added JavaScript file, and `scripts/**` is excluded from the packaged extension, so a script there would convert Word documents from a source tree and fail from an installed one.

Tiptap's own DOCX extensions were considered and are not used: both need a paid subscription and a private registry, and *import* runs on Tiptap's cloud with a JWT.

Both are `iife` and self-contained: a webview has no module loader and there is no CDN, so everything a surface needs travels with it. `npm run build-webview` writes both (`esbuild` + Sass) into `out/webview/`, which is ignored by git — so a source tree needs that one command before the editor or the Grid surface will open, and `vscode:prepublish` runs it for a packaged extension. No network and no `npm install` are needed *after* that build.

What neither surface owns: the document. Both send the whole file text to the extension, which applies it as a single `WorkspaceEdit` — so `Ctrl+Z`, the dirty dot, save, *Diff* and the git gutter keep working exactly as they do in the source editor.

Not built: a grid for `.xlsx`. It is a binary file that git cannot diff and that needs an `.xlsx` parser of its own (the maintained options are either deprecated on npm or drag in a pro tier), so a workbook keeps opening as what it is — a binary — rather than behind a grid that pretends it can be reviewed.

The Grid surface: a spreadsheet over CSV

`webview/grid.ts` is Univer, bundled the same way the page is; `src/gridEditor.ts` is its host. The contract is deliberately narrow: the host sends `{ type: 'set', rows }` with what `parseCsv` read, the surface sends back `{ type: 'change', rows }` once an edit settles, and the host writes that through `writeCsv` as one `WorkspaceEdit`. The cells are plain strings and the quoting is the extension's, so a one-cell change stays a one-line diff.

Four things about it are worth keeping written down, because each one cost an afternoon and none of them fails loudly:

- **The event is the workbook's.** In Univer 1.2.0 `onCommandExecuted` belongs to `FWorkbook`, not to the application. The first version guarded `typeof api.onCommandExecuted === 'function'`, which is false in every build, so the surface never reported anything: the grid accepted a cell edit, displayed it, and saved nothing.
- **A sheet is 1000 × 20 unless told otherwise.** `getRange` past the sheet's own size throws `Range is out of bounds`, so the read is clamped to `getMaxRows()` / `getMaxColumns()` and the workbook is created at least as large as the file it is given. Reading `Math.max(200, maxRows)` columns was asking for a range that did not exist.
- **Univer runs commands continuously** — selection, scroll, render. A debounce that clears its own timer on every command therefore never fires; the timer is armed once and allowed to run, and `flush()` decides whether anything really changed by comparing against the last thing it sent.
- **A throw inside a timer is invisible.** Reading the cells happened in the settle callback, so its exception became an unhandled page error and the surface simply stopped reporting. Both the read and the flush say so with `console.error`.

`src/cells.ts` holds the parts that need no browser — `asRows`, `widthOf`, `trimCells` — so `scripts/check-grid.mts` can prove the round trip without a window: a file opened and not edited reproduces itself byte for byte, and one edited cell stays one line.

The preview and the diff are **tabs of the editor group, not tabs inside one pane**. A custom editor that owned the document would have to bundle its own Monaco, and every copy of Monaco is worse than the one this window is already running: no git gutter, no search results landing on the right line, no other extension's decorations. The `Source | Preview | Diff` strip in the design is therefore three sibling tabs, which is also what VS Code's own Markdown preview does.

Which editor opens what:

File	Opens in
`.md`	**Temco Markdown**, by default — one *Open With… → Text Editor* away from the source, and the file never changes meaning
`.csv`, `.tsv`	the text editor — or the Grid, on request
`.xlsx`	whatever VS Code opens a binary with; there is no grid for it (see above)
everything else	untouched — the ordinary VS Code editor

One editor per kind, on purpose. There is no second rich-text surface for `.md`: the file is the source of truth and it is already readable in the surface's `Page` view, so an engine between the person and the bytes buys formatting they cannot review in a diff. The Office question is settled the same way, in `../../platform-design.md`:

`.docx` / `.xlsx` / `.pptx` are *not* edited in the product — Word documents come **in** through `src/docx.ts`, and an OOXML editor would make the source of truth binary, un-diffable, and (ONLYOFFICE being AGPL-3.0) unshippable.

The Markdown surface — removed, and where its features went

> **This surface is gone.** `temcoDocs.markdown`, `src/fluentEditor.ts` and `webview/docs/` were deleted, and `temcoDocs.page` became the default for `*.md`. What follows is kept as the record of what it did and why it went; the parts that survive live in `src/webviewText.ts` (the bridge) and `webview/shared/` (find, outline, the annotator).
>
> Why it went: two editors over one file is only justified when they *see* something different, and after the port they did not — the page had every authoring feature the Markdown surface had, so the second surface was a second place for a bug to live. Two things were lost with it and are recorded as lost: the `Editor | Split | Page` split view, and the "In this article" rail in preview. Neither is replaced by a fake: the native text editor *is* the source view (`View → Open source`), and navigation is the outline rail plus the Page pane.

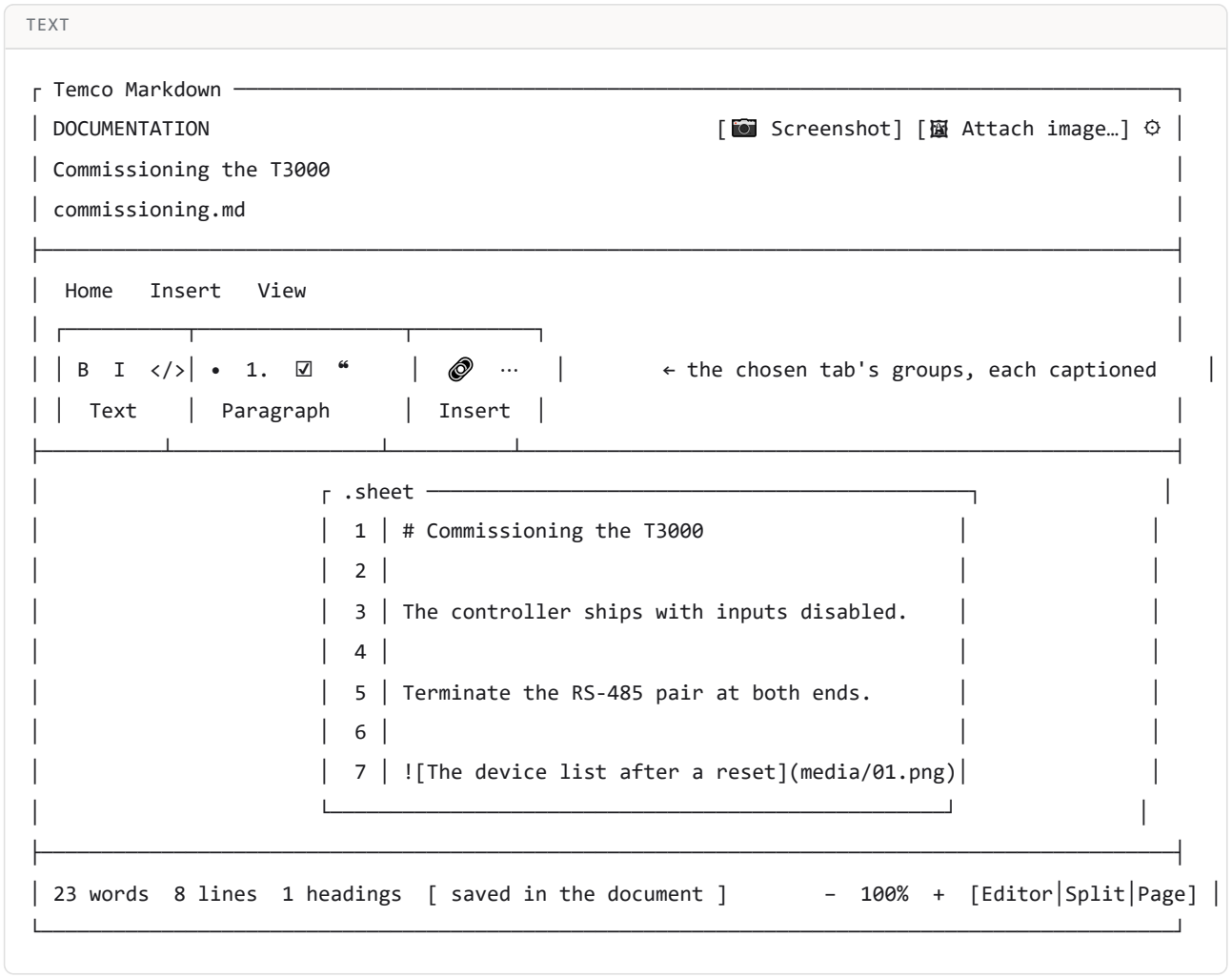
Feature it had	Where it is now
capture, attach, paste, caption, link-to-page	`src/webviewText.ts` — shared by the page, unchanged
annotate a picture	`webview/shared/annotate.tsx`, opened from the page's Insert tab
find	`webview/shared/find.tsx`, on the page's View tab (no replace: formatting boundaries)
outline	`webview/shared/outline.tsx` as a rail; `headingsIn()` in the page surface
preview / diff / source / properties	the page's View tab, through the same host actions
`Editor | Split | Page`	**lost** — the text editor is the source view
"In this article" rail	**lost** — the outline rail covers navigation

The original description is kept below, because the reasoning about chrome and vocabulary still applies to the page.

The page as a document, in **the application's own vocabulary**. It is the default editor for `*.md`, because authoring is what a page is for and the chrome around it is what was missing — not the engine. The document is still the file: the surface writes Markdown through the document, so `Ctrl+Z`, the dirty dot, save, *Diff* and the git gutter behave exactly as they do in the source editor, and *Open With… → Text Editor* shows the same bytes.

The layout is not invented here. `temco-ui/media/page.css` already carries a Markdown editor — `.md-editor` with a bar, a numbered gutter, `.md-line` colours and a status strip — and every other Temco page is drawn from the same file. This surface imports `page.css` into its bundle and renders `.view`, `.page-head`, `.md-editor`, `.badge` and `.segmented`, so it looks like the product instead of like an editor that happens to be inside it. Fluent supplies the popups only (the `⋯` menu, the outline popover, the prompt dialog); nothing on the page itself is a Fluent control.

TEXT



The arrangement is a document editor's, not a web page's: a head that says which document this is, a ribbon of named tabs whose chosen tab shows captioned groups of commands, the document itself on a **sheet** centred in the surface, and a status bar along the bottom with the counts on the left and the zoom and the view mode on the right.

Tab	Groups
Home	*Text* — bold, italic, inline code · *Paragraph* — bullet, numbered, task, quote · *Insert* — link, and `⋯` for every block
Insert	*Blocks* — table, code block, rule, `⋯` · *Links* — link to a page · *Page* — page properties
View	*Open* — the preview tab, the diff, the source editor · *Find* — the outline, find and replace

The tabs answer to `←` / `→` / `Home` / `End` and are a proper `tablist`: one tab in the tab order, `aria-selected` on the chosen one, `aria-controls` to its panel. A tab's controls exist **only** while its tab is chosen, which is why `check-docs` visits all three before it says every action is reachable.

Two sheets, or one. `Editor` shows the sheet alone; `Page` shows a sheet of `.article__body`; `Split` shows both with a divider you can drag. The reader **does not repeat the title** — the head already says it, and in `Split` the source says it too, so three copies of one line is two too many. Both sheets carry the same inset, so their first lines sit on the same pixel row. One pane keeps a 900px measure instead of running the width of the tab.

Zoom is a font size, not a transform. `– 100% +` in the status bar sets the *same* inline font size on the source and on its colour mirror, which is the only reason the colour stays under the words: two layers, one value. `MD-16` grows the text and measures both, because a size applied to one of them would look correct until the first character.

Every icon button carries an `aria-label` and a tooltip.

Action	How it is reached
bold · italic · inline code	button, and `Ctrl+B` / `Ctrl+I`; a backtick over a selection makes it code
heading 2 · 3 · 4	`···` — pressing the level again removes it
bullet · numbered · task · quote	one button each; every selected line is marked, and pressing again unmarks
code block · table · rule · five callouts	`···` — the code block asks for a language, the callout for one of the five DocFX renders
link	button, and `Ctrl+K` — the address is asked for, the selected words become the label
link to a page	`···` — the host offers the pages the registry declares and inserts a **relative `.md` path**, the only link the build can resolve
screenshot · attach image	the two buttons in the page head
page properties	the ⚙ in the page head, or `···` — the right-hand panel, for the page in front
outline	the outline button on the ribbon's View tab — the page's headings, indented by level, and one click puts the caret on one
find and replace	`Ctrl+F`, or the search button on the View tab: literal or pattern, a count, next/previous, and **replace all** as one edit
source · split · preview	the segmented control at the right of the status bar; the divider between the panes **drags**
preview · diff · source	the three icons beside it; each asks the host to run a command **with this document's URI**, and each is also in the tab menu's *Temco Docs* submenu

Writing, not just Markdown

Four behaviours exist because a page of instructions is written in a list, edited in place and read back:

Behaviour	Why
`Enter` inside a list starts the next item, numbering included, and `Enter` on an empty item ends the list	every step is typed as an item; making each one by hand is the most repeated keystroke in the page
`Tab` / `Shift+Tab` indent and outdent the selected lines	a nested step is two spaces, and nothing else should move
the colour behind the words	see below — the input stays a textarea, so none of the caret rules are put at risk for colour
the caret is revealed after a jump	the outline moves the caret into a line that may be off screen, and a caret you cannot see is a caret you do not trust

Colour without a second editor engine

The source pane is a `<textarea>`, and it has to stay one: every caret guarantee in this surface rests on the native element's own selection behaviour. So the highlighting is a **mirror `<pre>` behind the text**, and the textarea's own

text is transparent with only its caret and selection visible:

```
TEXT
```

```
┌──────────────────────────────────────────────┐
│ <pre class="editor__mirror" aria-hidden>  ← colour, tokens, identical font  │
│ <textarea class="source">                 ← transparent text, real caret     │
└──────────────────────────────────────────────┘
```

Two things make or break it, and both are asserted: the mirror must hold **exactly** the same characters (one dropped space slides the colour out from under the words), and the control must **not paint its own background** — Fluent gives the textarea a surface, which is precisely what hides the colour, so that one declaration is `!important` on purpose. `MD-10` in the scenario measures both, because neither is visible in a diff and both look fine in a type check.

File	What it owns
`src/fluentEditor.ts`	`MARKDOWN_VIEW_TYPE`, the `MarkdownEditor` subclass, `openMarkdown` — no protocol of its own
`src/webviewText.ts`	the bridge every webview surface shares: `set` / `change` / `pastedImage` / `capture` / `commitCapture` / `caption`, the echo guard, EOL handling, the capture command
`webview/docs/main.tsx`	the shell, the caret rules, the paste matrix (image → annotate, HTML table/list → Markdown)
`webview/docs/annotate.tsx`	the canvas annotator: arrow, box, ellipse, callout, redact, blur, numbered step, crop, undo
`webview/docs/markdown.ts`	`renderMarkdown` for the page, `htmlToMarkdown` for what is pasted in
`webview/docs/theme.ts`	`--vscode-*` → Fluent brand ramp, neutrals, radius and font; `watchTheme` repaints on a theme change
`webview/docs/docs.css`	geometry, the paper, the parts of a document Fluent has no concept of

The protocol

TEXT

```
webview                              host (WebviewTextEditor)
   │  {type:'ready'}                    │
   ├──────────────────────────────────▶│  answers with `set` — the document's own text
   │  {type:'change', text}             │
   ├──────────────────────────────────▶│  one WorkspaceEdit over the whole file, 350 ms after typi
   │  {type:'action', action:'capture'} │
   ├──────────────────────────────────▶│  takes the screenshot and holds it (nothing is written ye
   │  ◀──────── {type:'captureReady'} ──┤
   │  {type:'action', action:'commitCapture', caption}
   ├──────────────────────────────────▶│  writes media/NN-slug.png, records caption + version
   │  ◀── {type:'image', src, alt} ─────┤  answers with a page-relative address and the words
   │  {type:'action', action:'caption', src, caption}   (an image that landed without words)
   ├──────────────────────────────────▶│  records the words; nothing moves
   │  {type:'action', action:'preview' | 'diff' | 'source'}
   ├──────────────────────────────────▶│  runs the command **with this document's URI**
```

The capture is taken **before** the caption is asked for, and that order is deliberate: the screen a person wants is the one behind this window, and a dialog raised first would photograph the question. The bytes stay on the host while the surface asks, because a full screen is megabytes of base64 and it would cross the boundary twice for nothing.

`action` carries the document, because inside a custom editor there is no `activeTextEditor`: a command that fell back to "whatever editor is in front" would find nothing and quietly do nothing.

The caret, in the surface that ships

The three rules learned from the rich editor's round trip hold here too, and they are enforced in the webview rather than hoped for:

Rule	Why
a `set` exactly equal to what is on screen is dropped	the common case is an echo of our own write; redrawing identical text only moves the caret
a `set` that is an outside change while the surface has focus is **queued**, applied on blur	a replacement under the caret collapses the selection being dragged
an inserted image goes in **at the caret**, never appended	"quick" means where you are, not the end of the page
`ready` is posted **exactly once**, ever	the host answers `ready` with its own copy of the document; posting it again mid-write is what puts older text back under the caret

Validating this in real VS Code found a fourth defect that the prototype could not show, and it was the worst of the set: the message listener was re-subscribed whenever the status state changed, and each subscription posted `ready` again. `ready` makes the host answer with its own copy of the document — an answer that can arrive while a write of ours is still in flight, which puts the older text back **under the caret**. That is exactly the "the cursor comes to another

place" report, arriving through a channel that only exists in the real host. The listener is now installed once, and the current closure is carried by a ref.

`npm run check-docs` now guards it headlessly: the bundle is loaded into a DOM, a `set` is delivered, the surface is typed into, and the run fails if `ready` was posted more than once. Removing the fix makes it fail (`the surface reported ready`, `typing did not post a second ready`), which is the check earning its place — no screenshot and no type check can see this.

The status badge is part of the same fix. The host never echoes a change it applied itself, so a badge that compared the page against "the text the document opened with" stayed on *unsaved* forever. It now means what it says — `writing…` while the bridge is behind, `in document` once the edit is in the document — and it deliberately does not duplicate "is the file saved", which is the tab's dirty dot.

Screenshots, and why the capture is host-side

`getDisplayMedia` is not dependable inside a webview — the frame is not granted display capture in every build — so the capture happens where the OS window is. `captureScreen()` runs a small platform command (`System.Windows.Forms.Screen`

- `System.Drawing` on Windows, `screencapture -x` on macOS), writes a PNG to the temp folder, and **holds the bytes**. Linux reports that capture is unsupported and points at the paste.

Nothing else about the pipeline is different from a paste: the bytes are written beside the page and the surface gets a relative address back.

Concern	Decision
where an image lives	`media/` beside the page — one folder per page folder, so a page moved with its folder keeps its images
what the Markdown holds	`media/NN-slug.png`, relative — what the build resolves, so a published page needs no rewriting
the file name	`NN-slug`, the number continuing the folder's own sequence and the slug from the page — never `image.png`, and never a name that collides
two screenshots with the same name	impossible: the sequence number is read from the folder
what the file receives	nothing at capture time; the surface inserts the reference and the normal save path carries it in
the words	**one string**, used as the alt text and as the visible caption. A screen reader and a reader see the same sentence, and there is no second field to fall out of step
what a picture depicts	`appliesTo`, recorded at insert time (design §22.4, S1)

The pictures a version leaves behind

`media/attachments.json` sits beside the images and says what each one is:

JSON

```
{ "version": 1,
  "images": {
    "01-commissioning.png": {
      "caption": "The device list after a reset",
      "appliesTo": "13.0",
      "added": "2026-10-24"
    } } }
```

`appliesTo` is the version the release workflow's `version-report.json` names at the moment the image is inserted, so nothing has to be remembered by a person. When the report moves to a newer version, every picture still claiming the old one is **stale** — a warning, never a block (S3), and `temcoDocs.staleScreenshots` writes the report that records which ones were known about:

TEXT

```
# Screenshots a newer version has left behind — 2026-10-24

The documentation describes **13.1**. These 2 pictures were taken of something else,
and they still look true.

| Page | Image | Shows | Documentation |
| --- | --- | --- | --- |
| t3000/commissioning | 01-commissioning.png | 13.0 | 13.1 |
| t3000/data-points | 02-scaled.png | 13.0 | 13.1 |

Stale is a warning, not a block. Publishing one of these is allowed, and this report is what records
```

Two silences are deliberate. An image with **no** `appliesTo` is never called stale — "we do not know" is a different claim from "this is old", and a false accusation on every page written before the record existed would make the report worthless. And no version report means no report at all, because there is nothing to compare against.

What was proven in a real window

`.build/vscode-playwright-mcp/temco-markdown-editor.cjs` drives a dev build and passes **sixteen** steps: the surface opens, the source pane holds the file from disk, typing reaches the **file** after `Ctrl+S`, the caret is still where typing left it, re-opening the page costs **zero bytes**, the surface fills the editor area, a screenshot is taken and then asks what it shows, the caption becomes both the reference and the record, the preview renders it as a figure whose alt text is the same words, the colour behind the text lines up with the words, the outline lists the headings and jumps to one, find counts its matches, the view modes show one pane or both, and the tab menu's *Temco Docs* submenu offers the three views for this surface. The last one chooses each ribbon tab and proves the document is untouched by the choosing, because a tab is a place commands live, not a place text does. The sixteenth zooms in and measures the source against its colour mirror, then resets, because a zoom that moves one layer and not the other is invisible in every other check.

A second scenario, `.build/vscode-playwright-mcp/temco-page-editor.cjs`, drives the **rich page** and passes **three** steps: the page opens on the file that was in front and holds its text, typing marks the tab dirty and only `Ctrl+S`

writes the new line to the file as Markdown, and **Bold** from the ribbon produces `**` in the file while the heading it did not touch stays byte-identical. A fourth thing is proved outside the window: `npm run check-page` renders a whole page to HTML and back and requires it to come back unchanged, and the built bundle is loaded in a plain browser with a stubbed host to prove that the first answer to `ready` is not lost.

Three defects were found by running that scenario, and all three are fixed in the product:

Symptom	Cause	Fix
the editor tab was there and the area inside it was empty, with no webview at all	the bundle threw at load: `ColorBackground24Regular` was used but never imported, and `webview/page` was not in `tsconfig.webview.json`, so nothing type-checked it	the import is restored and the surface is type-checked like the other one
the page opened blank although the host had sent the text	the host answers `ready` at once, and the answer arrived while Tiptap was still creating its editor, where the handler dropped it	the answer is held in a module-level inbox and handed over when there is an editor to accept it
the scenario clicked an editor that was not on screen	two tabs carried the same file name — the source surface and the page — and a hidden webview still answers a selector	the page is left as the only tab, and "in front" is checked by the editor's own header naming *Temco Editor*

The first one is the lesson worth keeping: a webview that fails to start looks exactly like a webview that has not been caught up yet, and neither looks like an error. The `try` / `catch` in `resolveCustomTextEditor` exists because of it — a surface that cannot start now says so in its own tab.

A third scenario, `.build/vscode-playwright-mcp/temco-docs-page-link.cjs`, opens a page nested inside a documentation set. Its second step — **link to a page** and **page properties** — is reported as **blocked, not passed**: both read the project catalog, and the harness cannot register a project, so a folder holding `docs/help/registry.json` is still "Repositories with a registry: 0". Both menu items are reachable in the surface; the picker and the panel behind them are the unverified part, and registering a project in the harness is the missing capability.

Three defects were found by looking at that run's screenshots and logs rather than by its assertions, and all are fixed:

Symptom	Cause	Fix
the whole surface sat in a band at the top of the editor	`FluentProvider` renders its own `div` inside `#root`; with no height every `height: 100%` below it collapses to nothing	`#root > div { height: 100% }`
the source pane was 260 px tall in an 828 px pane	Fluent's `Textarea` is a **wrapper** around the real `<textarea>`, and the control it draws is capped by its own `max-height`	both levels fill: the wrapper is a flex box, the control takes `height: 100%` with `max-height: none`
a run failed with an empty source pane	the bar paints before the document arrives, so "the editor is open" was being decided by the first paint rather than by the text	the scenario waits for the file's text, which is what a person sees

The layout one is now an assertion (`MD-06`), because a screenshot alone does not keep it fixed.

Three harness facts are worth writing down, since they cost the most time: a webview is **two frames deep** (`iframe.webview` serves the platform's `index.html`, and the extension's document is the nested `#active-frame`); `retainContextWhenHidden` keeps a webview alive in a **background tab**, so finding its frame proves nothing about what a person can click; and on a loaded machine a cold dev build can take **two minutes** to activate, which is a timeout to raise rather than a bug to chase.

When a menu entry is gated on the editor, key it on the editor. Retiring the rich editor left three commands — *Open in Source Mode*, *Preview the Page*, *Diff the Page with HEAD* — gated on `activeCustomEditorId == 'temcoDocs.richTiptap'`, an id that no longer existed, so they were hidden and nothing failed. Two rules came out of it, and `MD-14` now enforces both:

Rule	Why
`activeCustomEditorId` names **this** surface; `editorLangId` does not	`activeCustomEditorId` is empty unless the active input *is* a custom editor, while `editorLangId` follows the active **text** editor — and a webview is not one. The built-in markdown preview keys its `editor/title/context` entry on `resourceLangId` for exactly this reason, and that difference is why the tab menu's Temco Docs submenu was invisible until it was corrected.
a `when` that can never be true looks exactly like a working feature	Nothing errors and nothing logs. Only an assertion that the command is *present* tells the two apart, which is why the step opens the menu instead of trusting the manifest.

The three now live in that submenu rather than in `editor/title`, because the surface already draws them in its own head — as icons, next to the view modes — and a title-bar twin both repeated them and squeezed the breadcrumb. `editor/title` keeps only the source-mode twins keyed on `editorLangId`, so each action appears once in each mode.

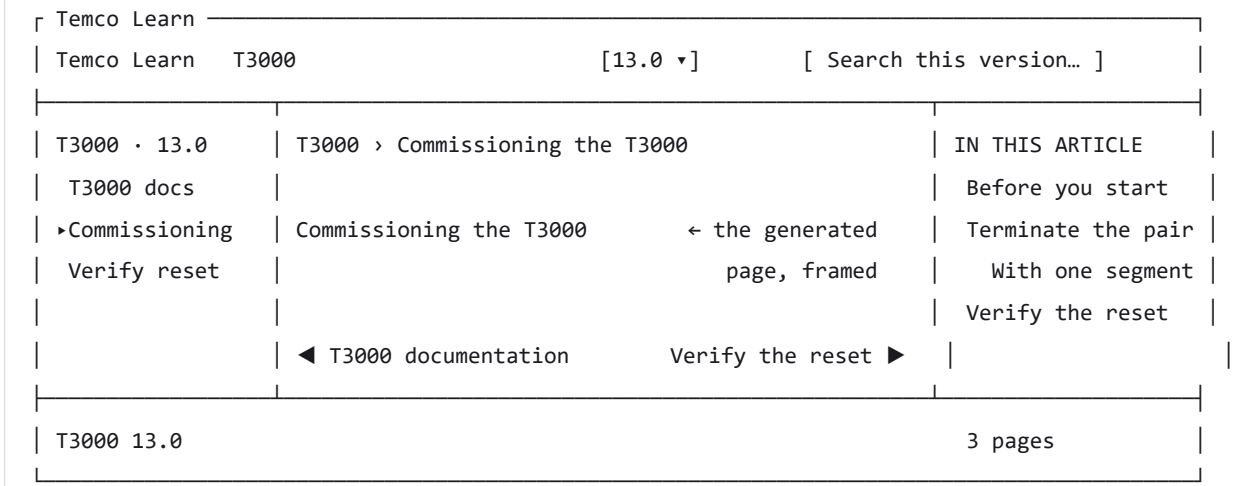
Two deliberate deviations

Deviation	Why
Fluent lives in `extensions/temco-docs/webview/docs/`, not in `temco-ui`	`temco-ui` has no React runtime and no JSX toolchain; extracting means giving it both. It is confined rather than unowned: `scripts/check-fluent.mts` fails the build if any other file imports `@fluentui`. The trigger to extract is a **second** Fluent surface.
`.md` opens in this editor by default	it is the authoring surface. The risk of a custom editor is that it changes what the file means; this one writes Markdown through the document, so it cannot. Every other surface here stays `option`.

The reader: what a published version looks like

Authoring is one half of documentation; this is the other half. A published version gets a page beside the files the generator produced, and **the reader is not a second renderer**: the article is the generated page in a frame, so DocFX (or MkDocs, or Docusaurus) keeps rendering its own Markdown, and the reader supplies the shape a documentation site has.

TEXT



Decision	Why
one self-contained file, model embedded as JSON	it works from a folder, from the loopback server, and inside a CHM alike — no fetch, no build step, no second file to keep in step
the article is a **frame**	a Markdown renderer here would be a second implementation of the generator's job, and it would drift
the sidebar is built from the **pages in the folder**	a doc set never has to be told about the reader, and the sidebar cannot disagree with what was published
"In this article" is read from the frame	it needs same-origin, which `siteServer` gives; from `file://` a browser refuses and the panel says nothing rather than guessing
written as `temco-reader.html`, never `index.html`	the generator's own home page is not ours to overwrite
the deep link is `#<href>`	a page can be sent to a colleague, and a reload lands on it

`src/learnReader.ts` is pure and `scripts/check-learn.mts` renders it: three columns, every page in the sidebar in order, the first page framed and marked, the version picker, no markup from a title, an empty set that still renders, and the hash deep link. Twenty checks, no browser.

The same shape inside VS Code. `Page` mode puts the article's own headings beside it — the rail a documentation site has, because a long page is crossed by its headings and not by scrolling. Headings are addressable: `renderMarkdown` writes `id` from `slugOf`, so a rail link lands on the section, `levels` indent, and the rail is hidden in `Split`, where the pane is already half a tab. `check-docs` asserts the links and their anchors, not the pixels.

The AI panel

TEXT

```
Temco Docs                                    (panel: the AI, docked on the right)
├─ header        the page's title (· stale) and the model in use · offline
├─ transcript    your question, the answer as it streams, notes, errors
├─ Draft update  → opens a diff: the page on the left, the draft on the right
│                  Apply replaces the whole page — one Ctrl+Z takes it back
├─ Ask the area  → the question, with the id, title and headings of the pages around it
└─ Ask           → a question about the page on screen
```

- **It writes nothing by itself.** A draft lands in a diff and in the panel; putting it into the page is a button, and the change is an ordinary undoable edit.
- **What reaches the model is fixed.** The page's text and metadata, and for an area question the ids, titles and headings of its neighbours. No other file, no repository sweep, no credential.
- **Every draft names its prompt version**, so a draft can be traced to the instructions that produced it.
- **An answer's citations are checked.** A page it names becomes a chip that opens it and is dashed when the page is stale; a name that is not a page is reported rather than believed; and an area answer that named nothing is replaced by search results instead of a synthesis.
- **With nothing installed the panel still opens** and says what to download, so the extension never looks broken — the AI is optional the whole way down.

The dashboard, and why this repository does not have one

`00-architecture.md` puts the dashboard on the **site**, not in the editor: a **generated static page** at `/_admin/` — coverage · stale by owner · docs-owed · unowned · unreviewed · broken links · feedback · health score — noindex, out of the PDF, the CHM and the offline zip, built by the product repository's own pipeline (`13-implementation-plan.md` step 1.19). Its rows deep-link **into** the editor, and that hand-off is this extension's half of it.

An earlier phase built an organisation read here instead — a repository list over the GitHub API and a sidebar view — which answered the dashboard's own question more weakly (repository-level facts, none of the page-level health) and in a second place. Both are deleted, along with the token setting they needed. The rule is the same one this document already carries for checks: **the extension never computes an answer the product repository already owns.**

Commands

Command	Does
`temcoDocs.showStatus`	refreshes, writes the full report to the *Temco Docs* output channel, offers **Show Problems**
`temcoDocs.validate`	same, and is what the status-bar item and the view title button call
`temcoDocs.refresh`	re-reads the registries
`temcoDocs.openRegistry`	opens `registry.json` for the folder the node belongs to
`temcoDocs.openPage`	opens the page, or offers to create the missing file
`temcoDocs.markdown`	opens the page in **Temco Markdown**, the Fluent authoring surface — the way back in from the source editor
`temcoDocs.staleScreenshots`	the report of pictures a newer version has left behind, from the record beside each page's images. Nothing is written and nothing is refused (design §22.4, S3)
`temcoDocs.reveal`	shows the page's file — or a chapter's, an area's or a source's folder — in the operating system's file manager (`revealFileInOS` , not the VS Code view of a similar name)
`temcoDocs.openTerminal`	opens an integrated terminal at the folder that row stands for
`temcoDocs.copyPath` · `copyRelativePath`	the row's path on the clipboard, absolute or relative to the open folder
`temcoDocs.newPage` · `newChapter`	a page (kind, title, file name) or a chapter — a folder with its first page, since an empty chapter does not exist
`temcoDocs.newPage.article` · `.task` · `.reference` · `.troubleshooting` · `.releaseNote`	the same as `newPage` with the kind already named, so the *New Page of a Kind* submenu is one click per kind
`temcoDocs.renamePage`	the title as a registry value, and the page's first `#` with it — never the file name
`temcoDocs.movePageUp` · `movePageDown`	a two-number swap of `order` in `registry.json`
`temcoDocs.retirePage`	`deprecated` and `superseded_by` ; the file stays, so old links keep redirecting
`temcoDocs.buildSite`	invokes the repository's own `docs:site` as a task and reports the files it produced
`temcoDocs.buildHelp`	invokes the non-deploying help build (`build:help:dry`) and reports the CHM it produced
`temcoDocs.stalenessReport`	opens the report of what is owed as an untitled Markdown document — nothing is written
`temcoDocs.newPage` · `newChapter` · the five kind commands	open the **New page page** (`src/newPagePage.ts`) with the row's place and, for a kind, its kind already chosen. The page writes through

Command	Does
	`writePage()` — one write, one confirmation that names the file
`temcoDocs.feedback`	reads each repository's `docs/help/feedback.json`, routes the rows to their owners, and calls out the aging

Where they surface: the **Workspace view's title bar** carries *New page*, *New chapter* and *Refresh* inline, and its `…` holds the page acts (open · reveal · terminal · copy a path · rename the title · move up · move down · retire · the *New Page of a Kind* submenu · the registry) plus Home and Validate — the same list the row's own right-click menu shows, which is the Library's shape and the Explorer's. A row's **left click** opens a page or expands a folder; a **right click** (or the keyboard's menu key) opens the menu; the hover carries nothing. Everything else is in the palette, where it costs no room — and `startRegistry` is a row in the view's empty state, so the view and the button that fixes it are never apart. There is no organisation-wide surface here by design: that question belongs to the site's generated dashboard, which this extension hands off to rather than re-implements.

What refreshes the view

- activation, and any workspace-folder change;
- a change to `**/docs/help/registry.json` (created, changed, deleted) — it changes the shape of the tree;
- a change to `**/docs/help/index.json` or `**/docs/help/page-meta.json` — they change the provenance the items display;
- an explicit refresh.

Everything else, including saving a page, does not refresh: the tree describes structure, not prose.

Rules the code follows

- **Two writes, each asked for.** Reading is what the extension does; it writes in exactly two places, and only on a command that names what it will create: the empty file for a page the registry already declares, and an empty registry where a folder you have open has none. Both are files the registry's own generator already expects, and neither is ever an existing file. **Nothing under `_admin/` or `_edit/` is written at all** — the design's rule is that those are generated pages, not documentation. What it downloads — the runtime, a model, the manifest that records them — goes to its own storage folder, or to the folder a setting names, and never into a repository.
- **A diagnostic lands where the fix is** — the page for a page problem, `registry.json` for a missing file or an unreadable registry.
- **Counts are not problems.** `reviewed: null` and empty tag lists are normal in a 335-page registry; as warnings they would bury the faults that matter.
- **No module knows about the shell.** Nothing reaches into the application's own code, by design.
- **Nothing is fetched unasked.** The only network calls are the runtime and model downloads the user starts from *Models and Runtime*: no telemetry, no update check, no background refresh, no organisation scan. No credential is stored at all.
- **Nothing native ships.** A compiled engine is per-OS; a downloaded one is not, which is what keeps one `.vsix` valid on every platform — the application, plain VS Code, Windows or macOS.
- **Building is not shipping.** The build commands invoke the repository's own scripts and report what those scripts produced. The form that deploys into a T3000 installation is never run, and nothing is uploaded or committed.

- **A generated page is never overwritten.** Which now means: nothing under `_admin/` or `_edit/` is written at all. The dashboard is the product repository's generated page; this extension is the deep-link target it hands off to.
- **An empty state is actionable.** A view that only explains itself is a dead end, and the folder with no registry is exactly where a new product repository starts. The Content view's empty state is a row that runs *Start a Registry* rather than a sentence telling the reader to go and find one.
- **A process, only while it is wanted.** The local server starts when something asks the AI for an answer, never at activation; it listens on loopback only; and it is stopped when idle, when its model is removed, and when the window closes. The extension holds no model in its own process, so nothing to load or unload blockingly.

Guardrails, in code

`12-ai-features.md`, *Guardrails*, lists them; this is where each one actually lives, so it can be checked rather than trusted.

Guardrail	Where it is enforced
No shipped page is edited by a model	A draft opens in a diff; the page changes only when **Apply draft** is pressed, and then as one undoable `WorkspaceEdit`
No secrets, no customer data	The only thing a prompt can take is a `PageContext` — page text, path, registry metadata — plus ids, titles and headings for an area question. No parameter exists that could carry another file, a token or a sweep of the repository
Traceable	`PROMPT_VERSION` is printed on every draft and in the panel's model line, and a draft also names its commit range (`temco-ai/src/ai/history.ts`)
Degradable	The panel opens with nothing installed and says what to download; the server never starts without a runtime and a model; a missing prompt file is a named error, not a stack trace; a page too large for one draft is refused rather than half-drafted
Deterministic first	Staleness is read from `index.json` , never recomputed; the low-confidence fallback is plain term overlap; citations are resolved against the registry
Cost bounded	6 turns of history, 40 area candidates, 40 commits, 100k characters of page text, 1024 tokens per answer and 2048 per draft, `--ctx-size 8192`
Offline	The runtime is a local process on `127.0.0.1` ; the only other network calls are the downloads the user starts
One content library	`scripts/check-fluent.mts` fails if anything outside `webview/docs` imports `@fluentui` — a second Fluent surface is the trigger to extract it, not a stray import
A surface cannot corrupt a file	every webview editor shares `WebviewTextEditor` , which owns the echo guard, the line endings and the single- `WorkspaceEdit` write; a surface only names its bundle, and `scripts/check-grid.mts` proves the CSV round trip keeps the file byte-for-byte when nothing is edited
One meaning per action	every formatting action is a pure transform in `src/markdownEdits.ts` , called by both editors — there is no second copy to drift
The bar cannot teach a key that does nothing	`scripts/check-shortcuts.mts` reads every `shortcut:` the ribbon advertises and traces it to an engine keymap, a `package.json` keybinding or a named shell listener; `Ctrl+F` and `Ctrl+Shift+X` were both taught and unbound when it was written
The page in front is a page, not a text editor	`frontUri()` reads the active tab as well as `activeTextEditor` , because a custom editor has none; without it Properties, the capture target and every row command silently saw nothing
Every image has words, and a version	`landImage()` is the only writer into `media/` ; it records the caption and the version the picture depicts, and `check-record` proves a picture with no recorded version is never called stale

Roadmap

Phase	Adds
1	**the Content view** — registry, checks, gaps, status bar
2	**the AI** — model download, runtime, draft page updates from a commit range, ask-the-area; a local model, offline, drafts only, a human accepts
3	**publishing** — invokes the product repo's site build and offline help build and hands over the artifacts; it orchestrates, it does not reimplement the pipeline
4	*(not a module of its own)* — the documentation health report is the platform's generated page at `/_admin/`
5	**feedback and health** — intake, and the report of what is owed; the report's cadence stays in the product repo's CI

The module's design is what this document and `registry-schema.md` hold; the platform-wide design is `design/temco-learn/`.

The AI is the reason the module exists. Everything before it is what the AI needs in order to know what it is talking about.